The APBS Electric-Field Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

1 What the calculator computes

The ApbsField calculator provides the electrostatic quantities used for the Buckingham electric-field contribution to nuclear shielding. Buckingham's idea is that an electric field at a nucleus can perturb the local electronic response, and the perturbed electrons then change the magnetic shielding felt by that nucleus. This implementation stops at the field-derived descriptor. It does not multiply by Buckingham response constants and it does not emit a shielding in ppm.

For each atom i, the code stores a field vector and a field-gradient matrix. The first is the solvated electric field

$$\boldsymbol{E}_i = (E_x, E_y, E_z) \tag{1}$$

in $\mathrm{V}/\text{Å}$. The second is a rank-2 electric-field-gradient matrix

$$G_{ab,i} = \frac{\partial E_a}{\partial r_b}(\boldsymbol{r}_i), \qquad a, b \in \{x, y, z\}, \tag{2}$$

in $\mathrm{V}/\text{Å}^2$. The row index a names the electric field component and the column index b names the coordinate direction in which the field is differentiated. The code symmetrizes this matrix and removes its trace before the spherical-tensor split, so the feature exported for $\boldsymbol{G}_i$ is the five-component traceless symmetric part. In this document, "kernel" means this uncalibrated field-derived tensor quantity: it is the geometric and electrostatic input that can correlate with a DFT shielding contribution after calibration.

2 Where shielding enters

For a nucleus in an applied magnetic field $\boldsymbol{B}_0$, the local magnetic field is written

$$\boldsymbol{B}_{\mathrm{nuc}} = (\boldsymbol{I} - \boldsymbol{\sigma})\boldsymbol{B}_0. \tag{3}$$

$\boldsymbol{I}$ is the 3×3 identity matrix. $\boldsymbol{\sigma}$ is the nuclear shielding tensor, a dimensionless linear-response coefficient: it tells how the electrons reduce or redirect the applied field at the nucleus. A matrix is needed because a molecule has orientation. A field applied along one laboratory axis can induce an electronic response whose field at the nucleus has components along several axes.

The electronic response in Eq. (3) is quantum mechanical. The calculator does not solve that response. Instead it supplies a classical electrostatic descriptor for one route by which the response changes. In an inhomogeneous protein environment, nearby charges, dielectric screening, and ions create an electric field at the nucleus. Buckingham-type expansions model the shielding perturbation as depending on that field and on how the field varies across the electronic cloud. A common schematic form is

$$\Delta\sigma_{\mathrm{iso}} \approx A_X E_{\parallel} + B_X E_{\parallel}^2, \qquad \Delta\boldsymbol{\sigma}_2 \approx \gamma_X \boldsymbol{G}_2. \tag{4}$$

X labels the nucleus or atom type; A_X, B_X, and γ_X are calibrated response coefficients carrying the units needed to convert field units into shielding units; $E_{\parallel}$, in $\mathrm{V}/\text{Å}$, is a chosen projection of the electric field, for example along a local bond axis; and $\boldsymbol{G}_2$, in $\mathrm{V}/\text{Å}^2$, is the traceless symmetric rank-2 part of the field gradient. Equation (4) explains why ApbsField keeps both a vector field and a rank-2 tensor. It is not the formula evaluated by this C++ calculator.

The APBS part matters because the electrostatic field is not the vacuum Coulomb sum over protein partial charges. The APBS solve includes a dielectric boundary and mobile-ion screening. ApbsField reads the resulting potential grid through the C bridge in `src/apbs_bridge.h`; the internal numerical method used by APBS is outside this calculator's model.

3 The tensor split used by the code

The shielding tensor in Eq. (3) and the field-gradient tensor in Eq. (2) are both rank-2 objects: each maps one direction index to another. The project stores such a 3×3 tensor $\boldsymbol{X}$ by a closed-form split into scalar, antisymmetric, and symmetric-traceless parts. The C++ owner of this convention is `SphericalTensor::Decompose` in `src/Types.cpp`.

The scalar part is

$$T_0 = \frac{1}{3} \operatorname{tr} \boldsymbol{X}. \tag{5}$$

It is the isotropic part: the average of the three diagonal entries. For a shielding tensor this is the part that survives rapid molecular tumbling and sets the isotropic shielding used in ordinary solution shifts after reference subtraction.

The antisymmetric part is stored as a three-component pseudovector,

$$\boldsymbol{T}_1 = \frac{1}{2} \begin{pmatrix} X_{yz} - X_{zy} \\ X_{zx} - X_{xz} \\ X_{xy} - X_{yx} \end{pmatrix}. \tag{6}$$

This is the transpose-odd part of the matrix written in vector form. It is usually not observed directly in standard isotropic NMR, but the representation keeps it when a calculator produces it.

The symmetric traceless matrix is

$$\boldsymbol{S} = \frac{\boldsymbol{X} + \boldsymbol{X}^{\mathsf{T}}}{2} - T_0 \boldsymbol{I}, \qquad \operatorname{tr} \boldsymbol{S} = 0. \tag{7}$$

It has five independent components. The code packs those components into the real spherical T_2 basis

$$\boldsymbol{T}_2 = \begin{pmatrix} \sqrt{2}\, S_{xy} \\ \sqrt{2}\, S_{yz} \\ \sqrt{3/2}\, S_{zz} \\ \sqrt{2}\, S_{xz} \\ (S_{xx} - S_{yy})/\sqrt{2} \end{pmatrix}. \tag{8}$$

The numerical factors in Eq. (8) are the code's normalization. They make the squared length of the five-vector equal to the Frobenius norm $\sum_{ab} S_{ab}^2$ of the underlying matrix. The full nine-double layout is

$$[T_0, T_{1x}, T_{1y}, T_{1z}, T_{2,-2}, T_{2,-1}, T_{2,0}, T_{2,+1}, T_{2,+2}]. \tag{9}$$

ApbsField stores the Cartesian field-gradient matrix on the atom, decomposes it with this routine, and writes the T_2 five-vector. Because the matrix has already been symmetrized and de-traced, T_0 and T_1 are structural zeros in the exported APBS field-gradient feature.

4 The method implemented

ApbsField depends on `ChargeAssignmentResult`. That upstream result places a partial charge q_j, in elementary charges, and a Poisson–Boltzmann radius a_j, in Å, on each atom from the active charge source. In the ff14SB flat-table path those values come from `data/ff14sb_params.dat`; AMBER topology and preloaded-charge paths can also provide the same fields. If a radius is missing, the APBS calculation returns failure. If the charge table reports placeholder PB radii, the code still fills the APBS fields but logs that the dielectric boundary is not physically validated.

The calculator forms separate coordinate, charge, and radius arrays and passes them to `apbs_solve`. The configured values in `data/calculator_params.toml` are

$$\begin{aligned} N_{\text{grid}} &= 161 \quad \text{points per axis}, \\ F_d &= \max(L_d + 40.0\,\text{Å},\, 40.0\,\text{Å}), \\ C_d &= F_d + 30.0\,\text{Å}, \\ \epsilon_{\text{protein}} &= 4.0, \qquad \epsilon_{\text{solvent}} = 78.54, \\ T &= 298.15\,\text{K}, \qquad I = 0.15\,\text{M}. \end{aligned} \tag{10}$$

L_d is the protein bounding-box extent along Cartesian direction d. The C++ layer computes both fine-grid lengths F_d and coarse-grid lengths C_d. The APBS bridge used here performs a single linearized Poisson–Boltzmann solve on the coarse box C_d, with single-Debye–Huckel boundary conditions, and returns a scalar potential grid ϕ in units of kT/e. The bridge also fixes the mobile ions as a $+1/-1$ pair at concentration I, with radii $0.95\,$Å and $1.81\,$Å, and passes a solvent probe radius of $1.4\,$Å to APBS. It does not return an electric field or a field gradient.

At an atom position $\boldsymbol{r}_i$, ApbsField first interpolates ϕ from the grid by trilinear interpolation, the eight-corner weighted average inside the cell containing the requested point. If the requested cell falls outside the returned grid, the interpolation helper returns 0.0; the padding values are intended to keep atom stencils inside the grid. Let h_a be the APBS grid spacing along axis a, and let $\hat{\boldsymbol{e}}_a$ be the unit vector in that direction. The electric field is then computed by a centered difference,

$$E_a(\boldsymbol{r}_i) = -\,\frac{\phi(\boldsymbol{r}_i + h_a\hat{\boldsymbol{e}}_a) - \phi(\boldsymbol{r}_i - h_a\hat{\boldsymbol{e}}_a)}{2h_a}. \tag{11}$$

The minus sign is the electrostatic convention $\boldsymbol{E} = -\nabla\phi$. Before unit conversion, Eq. (11) has units $kT/(e\,\text{Å})$.

The field-gradient matrix is another centered difference:

$$G_{ab}(\boldsymbol{r}_i) = \frac{E_a(\boldsymbol{r}_i + h_b\hat{\boldsymbol{e}}_b) - E_a(\boldsymbol{r}_i - h_b\hat{\boldsymbol{e}}_b)}{2h_b}. \tag{12}$$

This quantity has units $kT/(e\,\text{Å}^2)$ before conversion. Since $\boldsymbol{E} = -\nabla\phi$, $\boldsymbol{G}$ is the negative Hessian of the APBS potential. Some electrostatics texts use the opposite-sign Hessian as the electric field gradient; this document follows the code's $\partial E_a/\partial r_b$ convention. Equations (11) and (12) define the local finite-difference model the calculator keeps from the APBS potential. Near atom i, after symmetrization, the potential has the Taylor expansion

$$\phi(\boldsymbol{r}_i + \boldsymbol{s}) = \phi(\boldsymbol{r}_i) - \boldsymbol{E}_i \cdot \boldsymbol{s} - \frac{1}{2}\boldsymbol{s}^{\mathsf{T}}\boldsymbol{G}_i\boldsymbol{s} + O(|\boldsymbol{s}|^3). \tag{13}$$

The field vector is the local slope of the potential with the electrostatic minus sign; the matrix $\boldsymbol{G}_i$ records how that slope changes for small displacements. This quadratic patch summarizes the computation: ApbsField discards the absolute potential value and keeps the first- and second-derivative information that Buckingham-type models can use.

The raw finite-difference matrix is then projected in two steps,

$$\boldsymbol{G}^{\text{sym}} = \frac{\boldsymbol{G} + \boldsymbol{G}^{\mathsf{T}}}{2}, \qquad \widetilde{\boldsymbol{G}} = \boldsymbol{G}^{\text{sym}} - \frac{\operatorname{tr}\boldsymbol{G}^{\text{sym}}}{3}\boldsymbol{I}. \tag{14}$$

Symmetrization removes antisymmetric residue from interpolation and differencing; without it, the spherical decomposition would report a numerical T_1 pseudovector. The trace is the divergence of the electric field. For external sources in a charge-free neighborhood, Laplace's equation gives zero trace. The APBS grid also contains each atom's own Coulomb potential, whose point-source Laplacian is represented on the grid as a large finite trace near the atom. Subtracting $\operatorname{tr}\boldsymbol{G}/3$ from the diagonal removes that self-potential trace artifact and leaves the traceless tensor exported by the calculator.

Several guards are applied before storage. If the electric field contains a NaN or infinity, both $\boldsymbol{E}$ and $\widetilde{\boldsymbol{G}}$ are set to zero. If the APBS-native field magnitude exceeds $100.0\,kT/(e\,\text{Å})$, the field vector is rescaled to that magnitude; the field-gradient matrix is not rescaled by that guard. Non-finite entries of the field-gradient matrix are set to zero individually. Finally, both quantities are multiplied by the fixed thermal voltage

$$\frac{kT}{e} = 0.025\,693\,\mathrm{V} \tag{15}$$

from `src/PhysicalConstants.h`, matching the configured $298.15\,\mathrm{K}$, so the stored units become $\mathrm{V}/\text{Å}$ for $\boldsymbol{E}$ and $\mathrm{V}/\text{Å}^2$ for $\widetilde{\boldsymbol{G}}$.

The feature export follows the stored quantities. For N atoms, `apbs_E.npy` has shape $(N, 3)$ and stores the Cartesian electric field. `apbs_efg.npy` has shape $(N, 5)$ and stores the five T_2 components of $\widetilde{\boldsymbol{G}}$. If the APBS solve fails, `Compute` returns `nullptr`; the code does not substitute a vacuum Coulomb field.

5 External inputs and output

Two external ingredients feed this calculator. `ChargeAssignmentResult` provides per-atom partial charges and PB radii from the active force-field charge source, such as the ff14SB table or an AMBER topology; that source is outside the ApbsField calculation. APBS takes those arrays and solves the linearized Poisson–Boltzmann problem for the solvated potential; its solver internals are behind `src/apbs_bridge.h`. ApbsField consumes the returned potential grid, not APBS-internal fields or gradients, and derives $\boldsymbol{E}$ and $\widetilde{\boldsymbol{G}}$ itself. MOPAC charges, AIMNet2 charges or embeddings, and DSSP secondary-structure annotations do not enter this calculator.

The output is therefore an external-field descriptor: a solvated electric field and a traceless symmetric field-gradient kernel at each atom. It bears on shielding because Buckingham-type response terms can use the field vector and the T_2 field gradient as calibrated inputs. Before that calibration step, the numbers are in $\mathrm{V}/\text{Å}$ and $\mathrm{V}/\text{Å}^2$, not ppm, and they should be read as field-derived geometry rather than as predicted chemical shifts.